

Lezione 2: Proposizioni, dimostrazioni e regole logiche

Stefano M. Nicoletti

1 Lezione 2: Proposizioni, dimostrazioni e regole logiche

1.1 Scopo della lezione

Questa lezione riprende il ruolo del kernel e del typechecker, poi introduce il modo in cui Lean legge proposizioni e dimostrazioni. Il punto centrale è la corrispondenza tra proposizioni e tipi: una proposizione $P : \text{Prop}$ è un tipo, e una dimostrazione di P è un termine che abita quel tipo (Fonte: [Theorem Proving in Lean 4, Propositions and Proofs](#)).

Valuteremo esempi minimali per osservare, nel proof state, le regole di introduzione ed eliminazione dei connettivi logici e di alcuni costrutti specifici:

- implicazione $P \rightarrow Q$;
- congiunzione $P \wedge Q$;
- disgiunzione $P \vee Q$;
- negazione $\neg P$;
- bicondizionale $P \leftrightarrow Q$;
- `False`;
- principi classici espliciti.

1.2 Kernel e typechecker

Abbiamo visto che Lean permette di costruire dimostrazioni con tattiche, supporto di librerie, e tramite interfacce interattive. Questi strumenti aiutano l'utente, ma in ultima analisi, il kernel controlla l'oggetto formale prodotto e verifica che abbia il tipo richiesto (Fonte: [de Moura and Ullrich, Lean 4](#)).

Il typechecker controlla se un termine ha un certo tipo. Nelle dimostrazioni, il tipo atteso è la proposizione che vogliamo dimostrare. Un teorema viene accettato quando Lean riesce a controllare il termine di dimostrazione rispetto all'enunciato. Per questo, dal punto di vista del sistema, dimostrare significa costruire un termine ben tipato.

Questa prospettiva spiega perché il proof state è così utile. Il goal mostra il tipo che dobbiamo abitare. Le assunzioni mostrano i termini già disponibili nel contesto. Una tattica è un modo interattivo per trasformare questa situazione fino a chiudere tutti i goal.

1.3 Proposizioni come tipi

In Lean, $P : \text{Prop}$ dice che P è una proposizione. Se nel contesto compare $hP : P$, allora hP è una dimostrazione di P . Non è solo un'etichetta informale: è un oggetto formale che può essere usato per costruire altre dimostrazioni.

L'implicazione è l'esempio più diretto. Un termine di tipo $P \rightarrow Q$ si comporta come una funzione: prende una dimostrazione di P e restituisce una dimostrazione di Q . Se abbiamo

```
hPQ : P → Q
hP : P
```

allora $hPQ \ hP$ è una dimostrazione di Q .

1.4 Regole di introduzione ed eliminazione

Per ogni connettivo logico distinguiamo due domande.

- La regola di introduzione spiega come costruire una dimostrazione della proposizione composta.
- La regola di eliminazione spiega come usare una dimostrazione della proposizione composta per ottenerne delle componenti.

Guardare il goal significa chiedersi quale regola di introduzione può costruirlo. Guardare le assunzioni significa chiedersi quale regola di eliminazione può usarle. Questa terminologia segue la presentazione standard delle regole di deduzione naturale (Fonte: [Logic and Proof, Natural Deduction Rules](#)).

1.5 Implicazione

Per introdurre $P \rightarrow Q$, assumiamo temporaneamente P e costruiamo Q .

```
theorem lecture02_imp_intro (P Q : Prop) (hQ : Q) :
  P → Q := by
  intro hP
  exact hQ
```

La riga `intro hP` introduce l'assunzione temporanea $hP : P$. In questo esempio la conclusione Q è già disponibile come hQ , quindi possiamo chiudere il goal con `exact hQ`.

Per eliminare $P \rightarrow Q$, applichiamo l'implicazione a una dimostrazione di P .

```
theorem lecture02_imp_elim (P Q : Prop) :
  (P → Q) → P → Q := by
  intro hPQ
  intro hP
  have hQ := hPQ hP
  exact hQ
```

Qui $hPQ \ hP$ è applicazione di funzione: da $P \rightarrow Q$ e P otteniamo Q . Siamo di nuovo al modus ponens!

1.6 Congiunzione

Una congiunzione $P \wedge Q$ contiene entrambe le informazioni. Per introdurla, dobbiamo fornire una dimostrazione di entrambe le componenti (qui sotto le abbiamo inserite come assunzioni nell'enunciato).

```
theorem lecture02_and_intro (P Q : Prop) (hP : P) (hQ : Q) :  
  P ∧ Q := by  
  have hPAndQ := And.intro hP hQ  
  exact hPAndQ
```

La stessa regola `And.intro` può essere usata come tattica. Il goal $P \wedge Q$ si divide in due subgoal, uno per P e uno per Q .

```
theorem lecture02_and_intro_with_apply (P Q : Prop) (hP : P) (hQ : Q) :  
  P ∧ Q := by  
  apply And.intro  
  · exact hP  
  · exact hQ
```

Per eliminare una congiunzione usiamo le sue componenti, e preleviamo il congiunto sulla sinistra/destra.

```
theorem lecture02_and_elim_left (P Q : Prop) :  
  P ∧ Q → P := by  
  intro hPAndQ  
  exact hPAndQ.left  
  
theorem lecture02_and_elim_right (P Q : Prop) :  
  P ∧ Q → Q := by  
  intro hPAndQ  
  exact hPAndQ.right
```

1.7 Disgiunzione

Una disgiunzione $P \vee Q$ contiene una delle due informazioni. Per introdurla, basta fornire uno dei due lati (con `Or.inl`/`Or.inr`: injection left/right).

```
theorem lecture02_or_intro_left (P Q : Prop) :  
  P → P ∨ Q := by  
  intro hP  
  exact Or.inl hP  
  
theorem lecture02_or_intro_right (P Q : Prop) :  
  Q → P ∨ Q := by  
  intro hQ  
  exact Or.inr hQ
```

Per eliminare una disgiunzione dobbiamo ragionare per casi. Se abbiamo $P \vee Q$, non sappiamo in generale quale lato sia disponibile (in altri termini, non sappiamo se la disgiunzione sia vera

per via di P o di Q). Per ottenere una conclusione dobbiamo produrla in entrambi i rami: se riusciamo a concludere la stessa proposizione sia nel caso in cui sia P a essere vero, sia nel caso in cui lo sia Q , allora possiamo eliminare la disgiunzione. In questo caso, vogliamo concludere che $Q \vee P$ da $P \vee Q$: dobbiamo quindi ragionare valutando i due casi, P è vero oppure Q è vero, e per entrambi dimostrare che riusciamo a concludere $Q \vee P$.

```
theorem lecture02_or_elim (P Q : Prop) :
  P ∨ Q → Q ∨ P := by
  intro hPOrQ
  cases hPOrQ with
  | inl hP =>
    exact Or.inr hP
  | inr hQ =>
    exact Or.inl hQ
```

La stessa struttura si può scrivere con `Or.elim`.

```
theorem lecture02_or_elim_with_or_elim (P Q : Prop) :
  P ∨ Q → Q ∨ P := by
  intro hPOrQ
  apply Or.elim hPOrQ
  · intro hP
    exact Or.inr hP
  · intro hQ
    exact Or.inl hQ
```

1.8 False e negazione

False rappresenta uno stato contraddittorio. Se abbiamo una dimostrazione di False, possiamo chiudere qualunque goal.

```
theorem lecture02_false_elim (P : Prop) :
  False → P := by
  intro hFalse
  exact False.elim hFalse
```

In Lean la negazione è definita come implicazione verso False: $\neg P$ significa $P \rightarrow \text{False}$. Per eliminare una negazione, la applichiamo a una dimostrazione positiva di P .

```
theorem lecture02_not_elim (P : Prop) :
  P → ¬P → False := by
  intro hP
  intro hNonP
  exact hNonP hP
```

Per introdurre $\neg P$, assumiamo temporaneamente P e deriviamo False. Il seguente esempio è lo schema di ragionamento del modus tollens. `intro hP` svolge la seguente funzione: per dimostrare $\neg P$, assumiamo P e deriviamo False (una contraddizione). Osservate come cambia il goal:

```

theorem lecture02_not_intro (P Q : Prop) :
  (P → Q) → ¬Q → ¬P := by
  intro hPQ
  intro hNonQ
  intro hP
  have hQ := hPQ hP
  exact hNonQ hQ

```

Vediamone una versione istanziata con un esempio: se bere vino implica essere ubriachi, e non siamo ubriachi, allora non abbiamo bevuto vino. `intro hBevoVino` svolge qui la stessa funzione di `intro hP`.

```

theorem lecture02_vino_modus_tollens
  (BevoVino Ubriaco : Prop) :
  ((BevoVino → Ubriaco) ∧ ¬Ubriaco) → ¬BevoVino := by
  intro assunzione
  have hBevoUbriaco := assunzione.left
  have hNonUbriaco := assunzione.right
  intro hBevoVino
  have hUbriaco := hBevoUbriaco hBevoVino
  exact hNonUbriaco hUbriaco

```

1.9 Bicondizionale

Un bicondizionale $P \leftrightarrow Q$ contiene due implicazioni: una da P a Q e una da Q a P . Per introdurlo dobbiamo costruire entrambe e usare `Iff.intro`.

```

theorem lecture02_iff_intro (P Q : Prop) :
  (P → Q) → (Q → P) → (P ↔ Q) := by
  intro hPQ
  intro hQP
  apply Iff.intro
  · exact hPQ
  · exact hQP

```

Per eliminare un bicondizionale usiamo una delle due direzioni. In Lean `Iff.mp` è la direzione sinistra-destra (mp sta per *modus ponens*), mentre `Iff.mpr` è la direzione destra-sinistra (*modus ponens reversed*).

```

theorem lecture02_iff_elim_left (P Q : Prop) :
  (P ↔ Q) → P → Q := by
  intro hIff
  intro hP
  exact Iff.mp hIff hP

theorem lecture02_iff_elim_right (P Q : Prop) :
  (P ↔ Q) → Q → P := by
  intro hIff
  intro hQ

```

```
exact Iff.mpr hIff hQ
```

1.10 Fondazioni intuizioniste

La logica di base di Lean è costruttiva (intuizionista). Questo significa che una dimostrazione deve necessariamente costruire passo passo ciò che il goal richiede.

Alcuni principi familiari della matematica classica non sono disponibili automaticamente in questa base costruttiva:

- il terzo escluso, $P \vee \neg P$;
- l'eliminazione della doppia negazione, $\neg\neg P \rightarrow P$;
- la dimostrazione per assurdo in forma classica.

Questi principi possono essere usati in Lean, ma devono essere richiamati esplicitamente tramite la parte classica della libreria logica.

1.11 Contrapposizione

La direzione

```
(P → Q) → (¬Q → ¬P)
```

è costruttiva. Se abbiamo $P \rightarrow Q$ e $\neg Q$, allora per dimostrare $\neg P$ assumiamo P , otteniamo Q , e applichiamo $\neg Q$.

```
theorem lecture02_contrapposizione_avanti (P Q : Prop) :  
  (P → Q) → (¬Q → ¬P) := by  
  intro hPQ  
  intro hNonQ  
  intro hP  
  have hQ := hPQ hP  
  exact hNonQ hQ
```

La direzione inversa,

```
(¬Q → ¬P) → (P → Q)
```

richiede un ragionamento logico classico. Da $\neg Q \rightarrow \neg P$ e P non sappiamo costruire direttamente una dimostrazione di Q . Usiamo quindi il terzo escluso su Q con una dimostrazione per casi, tramite `cases Classical.em Q with`.

```
theorem lecture02_contrapposizione_classica (P Q : Prop) :  
  (¬Q → ¬P) → (P → Q) := by  
  intro hContrapp  
  intro hP  
  cases Classical.em Q with  
  | inl hQ =>  
    exact hQ
```

```
| inr hNonQ =>
  have hNonP := hContrapp hNonQ
  exfalse
  exact hNonP hP
```

Possiamo combinare le due direzioni in un bicondizionale.

```
theorem lecture02_contrapposizione_completa (P Q : Prop) :
  (P → Q) ↔ (¬Q → ¬P) := by
  apply Iff.intro
  · exact lecture02_contrapposizione_avanti P Q
  · exact lecture02_contrapposizione_classica P Q
```

Il punto teorico è che il bicondizionale completo usa una direzione costruttiva e una direzione classica: per la direzione classica bisogna specificare l'uso di costrutti classici, come il terzo escluso.

1.12 Dimostrazione per assurdo

Il terzo escluso è disponibile in Lean come `Classical.em P`, che produce una dimostrazione di $P \vee \neg P$. Questo permette di ragionare per casi su una proposizione arbitraria. Possiamo utilizzarlo nella classica dimostrazione per assurdo: assumere $\neg P$ porta a `False`, dunque classicamente concludiamo P .

```
theorem lecture02_proof_by_contradiction (P : Prop) :
  (¬P → False) → P := by
  intro hContraddizione
  cases Classical.em P with
  | inl hP =>
    exact hP
  | inr hNonP =>
    exfalse
    exact hContraddizione hNonP
```

Lean fornisce anche il principio classico dedicato tramite `Classical.byContradiction`.

```
theorem lecture02_by_contradiction_classical (P : Prop) :
  (¬P → False) → P := by
  intro hContraddizione
  exact Classical.byContradiction hContraddizione
```

Nota teorica: questo passaggio non è equivalente alla negazione costruttiva. La regola costruttiva $P \rightarrow \neg P \rightarrow \text{False}$ dice che P e $\neg P$ sono incompatibili, e generano una contraddizione. La conclusione classica per assurdo $(\neg P \rightarrow \text{False}) \rightarrow P$, invece, elimina una doppia negazione e costruisce P .

1.13 Esempi di argomentazione

Dopo queste regole minimali, torniamo agli argomenti in linguaggio naturale con più consapevolezza.

```

-- Linguaggio naturale: se abbiamo un'assunzione, se dall'assunzione segue
-- una conseguenza, e se dalla conseguenza segue una conclusione, allora
-- la conclusione.
example (Assunzione Consequenza Conclusione : Prop) :
  (Assunzione  $\wedge$  (Assunzione  $\rightarrow$  Consequenza))  $\wedge$ 
  (Consequenza  $\rightarrow$  Conclusione)  $\rightarrow$ 
  Conclusione := by
intro hArgomento
have hPrimoPasso := hArgomento.left
have hAssunzione := hPrimoPasso.left
have hPasso1 := hPrimoPasso.right
have hPasso2 := hArgomento.right
have hConsequenza := hPasso1 hAssunzione
have hConclusione := hPasso2 hConsequenza
exact hConclusione

```

Il secondo usa una disgiunzione:

```

-- Linguaggio naturale: se piove oppure nevica, e in ciascuno dei due casi
-- prendo l'ombrello, allora prendo l'ombrello.
example (Piove Nevica PrendoOmbrello : Prop) :
  ((Piove  $\rightarrow$  PrendoOmbrello)  $\wedge$ 
  (Nevica  $\rightarrow$  PrendoOmbrello))  $\wedge$ 
  (Piove  $\vee$  Nevica)  $\rightarrow$ 
  PrendoOmbrello := by
intro hArgomento
have hRegole := hArgomento.left
have hPioveONevica := hArgomento.right
have hPioveOmbrello := hRegole.left
have hNevicaOmbrello := hRegole.right
-- Non sappiamo quale lato della  $\vee$  è valido, quindi consideriamo entrambi
-- i casi.
cases hPioveONevica with
| inl hPiove =>
  exact hPioveOmbrello hPiove
| inr hNevica =>
  exact hNevicaOmbrello hNevica

```

La versione con `Or.elim` rende esplicita la struttura astratta della regola: da una dimostrazione di $A \vee B$, una funzione $A \rightarrow C$ e una funzione $B \rightarrow C$, otteniamo C .

```

-- La stessa dimostrazione, usando direttamente `Or.elim`.
example (Piove Nevica PrendoOmbrello : Prop) :
  ((Piove  $\rightarrow$  PrendoOmbrello)  $\wedge$ 
  (Nevica  $\rightarrow$  PrendoOmbrello))  $\wedge$ 
  (Piove  $\vee$  Nevica)  $\rightarrow$ 
  PrendoOmbrello := by
intro hArgomento
have hRegole := hArgomento.left
have hPioveONevica := hArgomento.right
have hPioveOmbrello := hRegole.left

```



```

have hNevicaOmbrello := hRegole.right
apply Or.elim hPioveONevica
· intro hPiove
  exact hPioveOmbrello hPiove
· intro hNevica
  exact hNevicaOmbrello hNevica

```

Un ultimo esempio:

```

-- Linguaggio naturale: se la fonte è autentica oppure i dati sono coerenti,
-- e ciascuno dei due casi supporta la tesi, e se una tesi supportata rende
-- plausibile la conclusione, allora la conclusione è plausibile.
example (FonteAutentica DatiCoerenti TesiSupportata ConclusionePlausibile :
  Prop) :
  ((FonteAutentica v DatiCoerenti) ∧
   ((FonteAutentica → TesiSupportata) ∧
    (DatiCoerenti → TesiSupportata))) ∧
  (TesiSupportata → ConclusionePlausibile) →
  ConclusionePlausibile := by
intro hArgomento
have hPrimoPasso := hArgomento.left
have hFonteODati := hPrimoPasso.left
have hRegoleSupporto := hPrimoPasso.right
have hFonteSupporta := hRegoleSupporto.left
have hDatiSupportano := hRegoleSupporto.right
have hSupportoConclude := hArgomento.right
have hTesi :=
  Or.elim hFonteODati hFonteSupporta hDatiSupportano
have hConclusione := hSupportoConclude hTesi
exact hConclusione

```

1.14 Tattiche, comandi e scorciatoie

In questa lezione abbiamo usato soprattutto tattiche che corrispondono alle regole di introduzione ed eliminazione dei connettivi.

- `intro`: introduce un'assunzione temporanea quando il goal è un'implicazione o una negazione;
- `exact`: chiude il goal fornendo un termine del tipo richiesto;
- `have`: introduce un risultato intermedio nel contesto;
- `apply`: applica una regola, un teorema o un costruttore al goal corrente;
- `cases`: distingue i casi di una disgiunzione;
- `ex falso`: cambia il goal corrente in `False`, quando vogliamo chiudere il goal tramite una contraddizione.

Abbiamo inoltre usato alcuni costruttori ed eliminatori espliciti:

- `And.intro`, `.left`, `.right` per la congiunzione;

- `Or.inl`, `Or.inr`, `Or.elim` per la disgiunzione;
- `False.elim` per usare una contraddizione;
- `Iff.intro`, `Iff.mp`, `Iff.mpr` per il bicondizionale;
- `Classical.em` per il terzo escluso;
- `Classical.byContradiction` per la dimostrazione classica per assurdo.

1.15 Esercizi e soluzioni

Trovate i file con esercizi e soluzioni (`Exercises.lean` e `Solutions.lean`) sul sito di riferimento.

1.16 Fonti e letture

- Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich, *Theorem Proving in Lean 4*, capitolo "Propositions and Proofs": https://lean-lang.org/theorem_proving_in_lean4/Propositions-and-Proofs/.
- Jeremy Avigad, Robert Y. Lewis, Floris van Doorn, *Logic and Proof*, appendice "Natural Deduction Rules": https://leanprover.github.io/logic_and_proof_lean3/n_d_quickref.html.
- Leonardo de Moura and Sebastian Ullrich, "The Lean 4 Theorem Prover and Programming Language": <https://lean-lang.org/papers/lean4.pdf>.